

Task 3 — Nationality Detection System

Ethnicity Classification, Emotion Recognition, Age Estimation and Dress Colour Detection

Machine Learning Project Report

Custom Keras CNN · DeepFace · OpenCV · scikit-learn · Streamlit

Contents

1. Problem Statement
2. Understanding the Problem
3. System Overview
4. Datasets and Models Used
5. Component 1 — Ethnicity / Nationality Classification
6. Component 2 — Emotion Recognition (DeepFace)
7. Component 3 — Age Estimation (reused from Task 1)
8. Component 4 — Dress Colour Detection (Classical CV)
9. The Combined Decision Logic
10. The Streamlit GUI
11. Challenges Faced
12. Results Summary
13. Conclusion and Future Work

1. Problem Statement

The task is to build a nationality detection system that accepts a face photograph, identifies the person's ethnicity (used here as a proxy for nationality, since nationality is not a visually inferable property), and then applies a set of conditional rules that decide which additional attributes to predict and display. The four rules are:

- Indian — predict emotion, age, and dress colour.
- United States (White) — predict emotion and age only.
- African — predict emotion and dress colour only.
- Other nationality — predict ethnicity and emotion only.

The system must also provide a proper graphical user interface with an image preview and a dedicated output section showing all results for the predicted category.

2. Understanding the Problem

Reading the task carefully reveals that the word “nationality” is used loosely. Nationality is a legal concept tied to citizenship documents; it cannot be inferred from a face image. What can be inferred, imperfectly, is the person's apparent ethnic or regional background. The system therefore classifies broad ethnicity categories and maps them to the four rules in the task. This distinction is noted clearly because it matters for understanding both what the model can and cannot do and what the results actually mean.

Once that reframing is in place, the task decomposes into four independent sub-problems:

- Ethnicity classification from a face image — to decide which rule to apply.
- Emotion recognition from a face image — required in every branch.
- Age estimation from a face image — required for the Indian and US branches.
- Dress colour detection from the region below the face — required for the Indian and African branches.

No single model solves all four. The solution is a pipeline: the components run in sequence and their outputs are merged by a short decision function that implements the branching logic. Identifying this structure early was the key design decision, because it turned a single ambiguous requirement into four clear, independently solvable sub-tasks.

3. System Overview

The system is a four-component pipeline wrapped inside a Streamlit web application. When a user uploads a photograph, the face is located and cropped, the relevant models are run, and their outputs are merged by the decision layer before anything is shown on screen. The table below summarises the components and, importantly, distinguishes what was self-built from what was integrated from an existing library.

Component	How it works	Built or integrated
Ethnicity / nationality	Custom Keras CNN (race head) + skin-tone refinement	Self-built model
Emotion	DeepFace.analyze (pretrained)	Integrated library
Age	Keras CNN reused from Task 1	Self-built (Task 1)
Dress colour	OpenCV + scikit-learn KMeans + LAB naming	Self-built (classical CV)
Decision logic / GUI	Plain Python + Streamlit	Self-built



Figure 1 — the four-component pipeline: input → face crop → ethnicity, emotion, age and dress-colour components → decision logic → branch-specific output.

4. Datasets and Models Used

4.1 UTKFace (ethnicity + age training data)

The ethnicity classifier and the age model are both built on UTKFace, a large public face dataset of over twenty thousand images in which the age, gender and ethnicity of each person are stored directly in the filename. UTKFace labels ethnicity in five classes — White, Black, Asian, Indian and Others — which is exactly the label set this project uses (the `race_map` in the code maps indices 0–4 to these five names). Parsing labels straight from the filename made loading the data straightforward, and the five-class scheme maps cleanly onto the task's four branches.

For the branch rules, the five UTKFace classes are mapped as follows: Indian maps to the Indian branch, Black/African maps to the African branch, White maps to the US branch, and the remaining classes fall

into the Other branch. This mapping is a deliberate simplification recorded as a known limitation — a person of any given apparent ethnicity may hold any national citizenship.

4.2 The emotion model is not trained here — it is DeepFace

It is important to be accurate about this: no emotion model was trained for this project. Emotion recognition is handled entirely by DeepFace, an open-source facial-analysis library that ships with its own pretrained emotion classifier. The project integrates DeepFace rather than building an emotion network from scratch. DeepFace's emotion model returns one of seven labels — happy, sad, angry, fear, disgust, surprise and neutral. Section 6 describes exactly how it is called.

5. Component 1 — Ethnicity / Nationality Classification

5.1 The model

Ethnicity is predicted by a custom Keras convolutional model (loaded from `age_gender_race_model.keras`) that produces a race output over the five UTKFace classes. The model is loaded once and cached with Streamlit's `@st.cache_resource` so it is read from disk only at start-up rather than on every prediction.

```
@st.cache_resource
def get_models():
    age_gender_model = load_model('best_model.keras')          # Task 1
    nationality_model = load_model('age_gender_race_model.keras')
    return age_gender_model, nationality_model

race_map = {0:'White', 1:'Black/African', 2:'Asian',
            3:'Indian', 4:'Others'}
```

5.2 Preprocessing and skin-tone refinement

Before the model runs, the face is cropped with an OpenCV Haar cascade and normalised: white-balanced, converted to grayscale, resized to 128×128, and contrast-enhanced with CLAHE. This normalisation makes the classifier more robust to lighting differences between photos.

The model's raw race probabilities are then passed through a small refinement step. The African and Indian classes are the easiest to confuse, so when the model's top prediction is one of those two and the gap between them is small, the system measures the actual skin-tone brightness from the centre of the face and uses it to break the tie — darker toward African, lighter toward Indian. This is a deliberate hand-built correction layered on top of the model's output to address its single most common confusion.

```
def refine_with_skin_tone(probs, pil_face):
    top = int(np.argmax(probs))
    if top in (AFRICAN, INDIAN) and abs(probs[INDIAN]-probs[AFRICAN]) < 0.6:
        brightness = mean(get_skin_tone(pil_face))
        return AFRICAN if brightness < THRESHOLD else INDIAN
    return top
```

The refined class is then mapped to a nationality branch (Indian → Indian, Black/African → African, White → US, everything else → Other), and the model's softmax probability for that class is shown as a confidence percentage.

5.3 Results

Evaluated on a held-out test set, the ethnicity model reaches roughly 76% overall accuracy. The per-class results are uneven: White, Black/African and Asian are recognised well (F1 around 0.81–0.85), Indian is moderate, and the catch-all “Others” class is weakest (F1 around 0.36), which is expected since it groups together several visually different groups. The confusion matrix below shows where the errors concentrate.

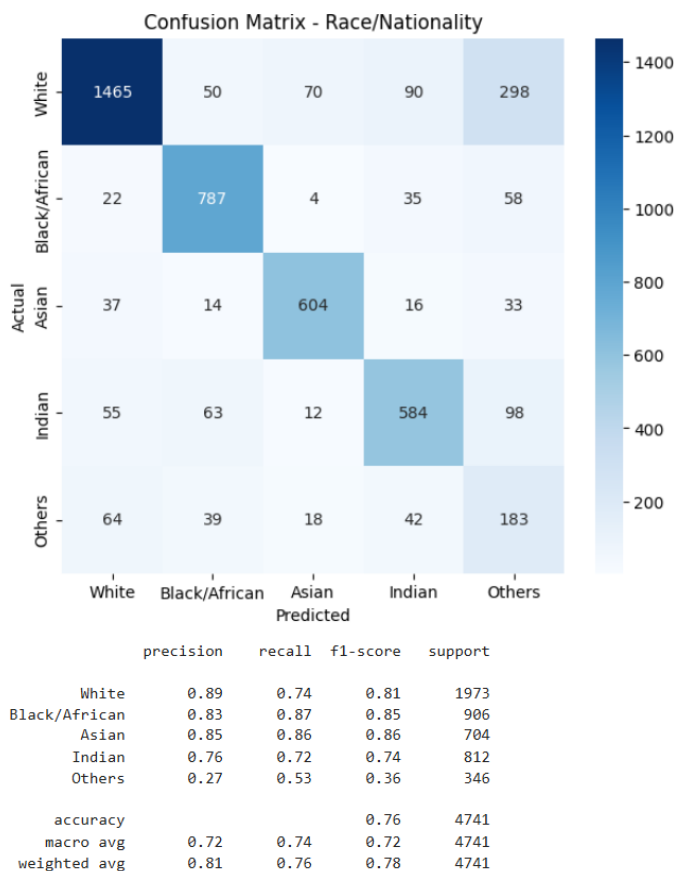


Figure 2 — confusion matrix and classification report for the five-class ethnicity model on the test set. The “Others” class is the hardest, and the Indian / Black confusion is what the skin-tone refinement step is designed to mitigate.

6. Component 2 — Emotion Recognition (DeepFace)

6.1 Architecture

Emotion recognition is performed by DeepFace, a pretrained open-source facial-analysis library, not by a model trained for this project. When a face image reaches the emotion function it is converted from RGB to BGR (the ordering OpenCV and DeepFace expect) and passed to DeepFace.analyze with the emotion

action. Internally, DeepFace runs the image through its own pretrained convolutional network and returns probability scores over seven emotion classes, of which the dominant one is taken.

```
def predict_emotion(pil_image):
    img = cv2.cvtColor(np.array(pil_image.convert('RGB')), cv2.COLOR_RGB2BGR)
    try:
        result = DeepFace.analyze(img, actions=['emotion'],
                                   enforce_detection=False)
        return result[0]['dominant_emotion'].lower()
    except Exception:
        return 'neutral'
```

Two design choices are worth noting. The `enforce_detection=False` flag tells DeepFace to proceed even when it cannot cleanly find a face, which stops the app crashing on already-cropped or difficult images. And the whole call is wrapped in a `try/except` that falls back to 'neutral' on any error, so a failure in this one component never breaks the rest of the pipeline. The returned label is then mapped to a colour and emoji for display in the interface.

To be completely clear: this component is integrated, not self-built. The emotional classification is done by DeepFace's pretrained model; what is my own here is the preprocessing, the safe error handling, and the way the result is presented. This is an honest and common way to add a capability that would otherwise require its own dataset and training run.

7. Component 3 — Age Estimation (reused from Task 1)

Age estimation reuses `best_model.keras` from Task 1 without any retraining. That model was trained on UTKFace to regress a continuous age from a grayscale 128×128 face crop. In Task 3 the age is only computed when the predicted branch is Indian or US, exactly as the rules require. The preprocessing is kept identical to Task 1 — grayscale, resized to 128×128, scaled to 0–1 — because feeding the model anything else would give meaningless results.

```
age_input = preprocess(pil_image)          # grayscale 128x128, /255
ag = age_gender_model.predict(age_input)
gender = 'Male' if ag[0][0][0] < 0.5 else 'Female'
age     = int(ag[1][0][0])
```

Because age from a face carries a few years of error, the interface displays the estimate with a “± 6 years” note rather than implying false precision.

8. Component 4 — Dress Colour Detection (Classical CV)

8.1 Why classical CV instead of a model

Dress colour does not need a neural network. The question — what is the dominant clothing colour — is answered directly by clustering the pixels below the face and naming the most prominent cluster. This is faster, needs no training data, and is easy to reason about.

8.2 Implementation

The clothing region is taken as the band of pixels below the detected face. That region is white-balanced, then filtered to remove very bright/near-white background pixels and skin pixels (using a YCrCb skin test), so the clustering focuses on actual clothing. The remaining pixels are clustered with scikit-learn's KMeans, and each cluster is scored by a combination of how saturated (colourful) it is and how many pixels it contains — so a small but vivid garment isn't lost to a large dull background.

```
km = KMeans(n_clusters=min(3,len(filtered)), n_init=10,
            random_state=42).fit(filtered)
hsv = rgb_to_hsv(km.cluster_centers_)
scores = hsv[:,1] * np.sqrt(cluster_counts) # saturation x size
dominant = centers[np.argmax(scores)]
```

8.3 Colour naming

The dominant cluster's RGB value is converted to a human-readable colour name by finding the nearest reference colour in CIELAB space, which matches human colour perception more closely than a plain RGB distance. The reference palette covers seventeen common colours (black, white, gray, red, maroon, pink, orange, brown, tan, beige, yellow, olive, green, teal, blue, navy, purple).

```
def closest_color_name(rgb):
    px = rgb_to_lab(rgb)
    return min(COLOR_NAMES, key=lambda n: lab_distance(px, COLOR_NAMES[n]))
```

9. The Combined Decision Logic

This is the part that answers the problem statement. The ethnicity classifier (after skin-tone refinement) decides the branch, and a short routing function then runs only the components that branch requires. Emotion is the one thing computed in every branch; age and dress colour are conditional.

```
result = {'nationality': nationality, 'emotion': predict_emotion(img), ...}

if nationality == 'Indian':
    result['age'] = age
    result['dress'] = detect_dress_color(img)
elif nationality == 'US':
    result['age'] = age
elif nationality == 'African':
    result['dress'] = detect_dress_color(img)
# 'Other' -> ethnicity label + emotion only
```

Walking through it makes the logic transparent. Emotion runs for everyone. The Indian branch adds both age and dress colour; the US branch adds only age; the African branch adds only dress colour; and the Other branch simply reports the ethnicity label alongside emotion. Each component does one job, and this short function is where the actual task rule is satisfied — which matters because the brief weights correct logic as heavily as model accuracy.

10. The Streamlit GUI

The interface was built with Streamlit, the same as the earlier tasks, because it turns a plain Python script into a clean web app with very little extra code. It is a two-column layout: the left side handles image upload and preview, and the right side shows a results panel whose contents change with the predicted branch. The whole app is styled with a custom dark theme so the output cards read clearly.

The examples below show the app running on real uploads, demonstrating each branch's behaviour — the conditional cards (emotion, age, dress colour, ethnicity) appearing or not according to the detected nationality, along with the race-model confidence bar.

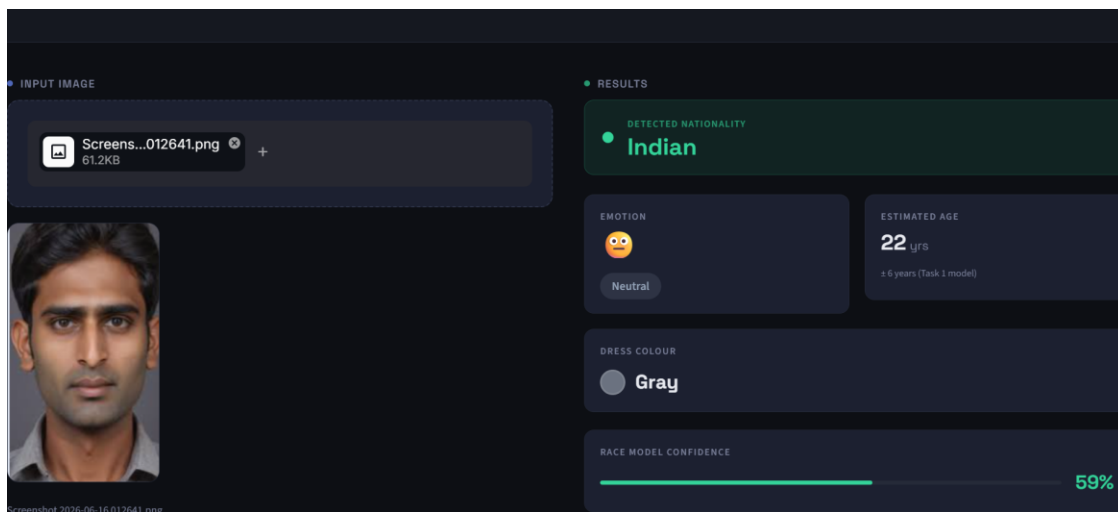


Figure 3 — Indian branch: emotion, estimated age and dress colour all shown, with the race-model confidence below.

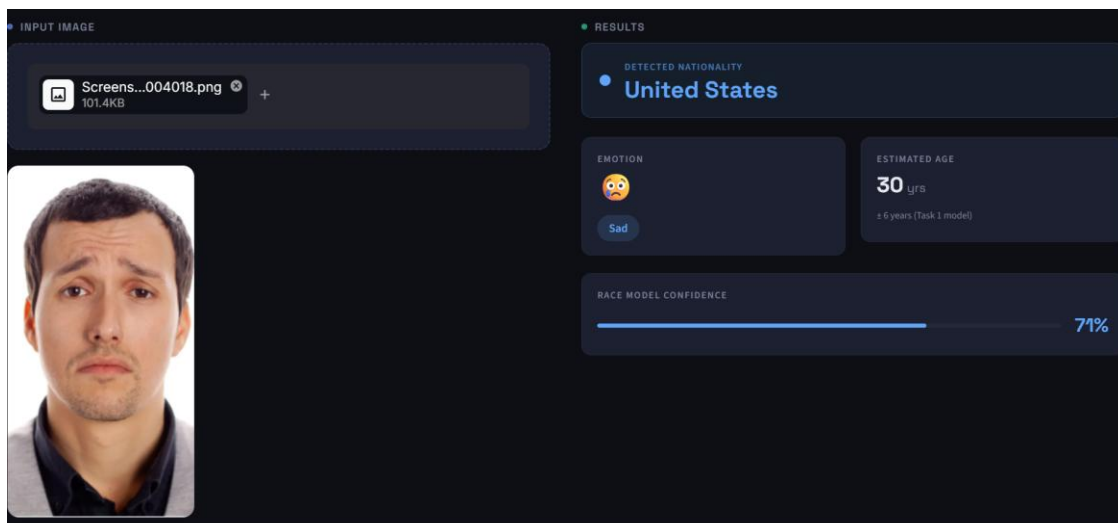


Figure 4 — a second run showing the conditional results panel for a different branch.

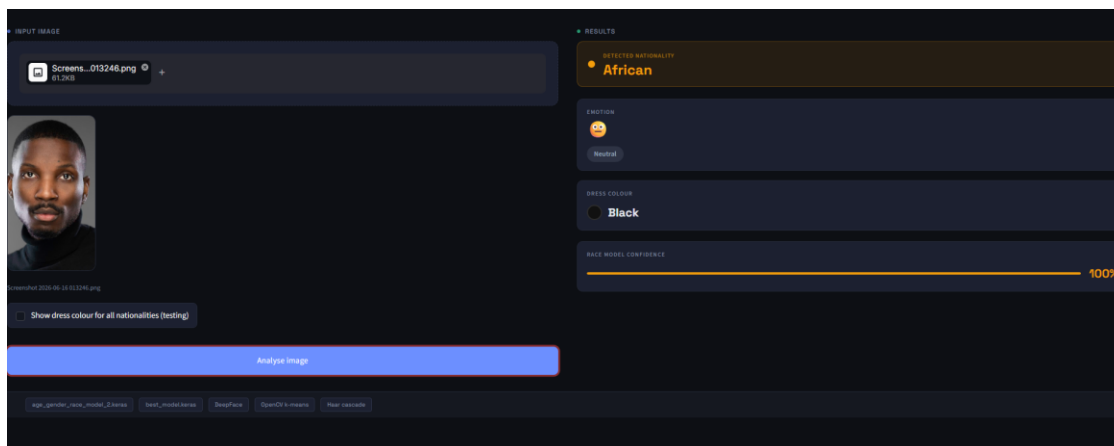


Figure 5 — the interface adapting its output cards to the detected nationality.

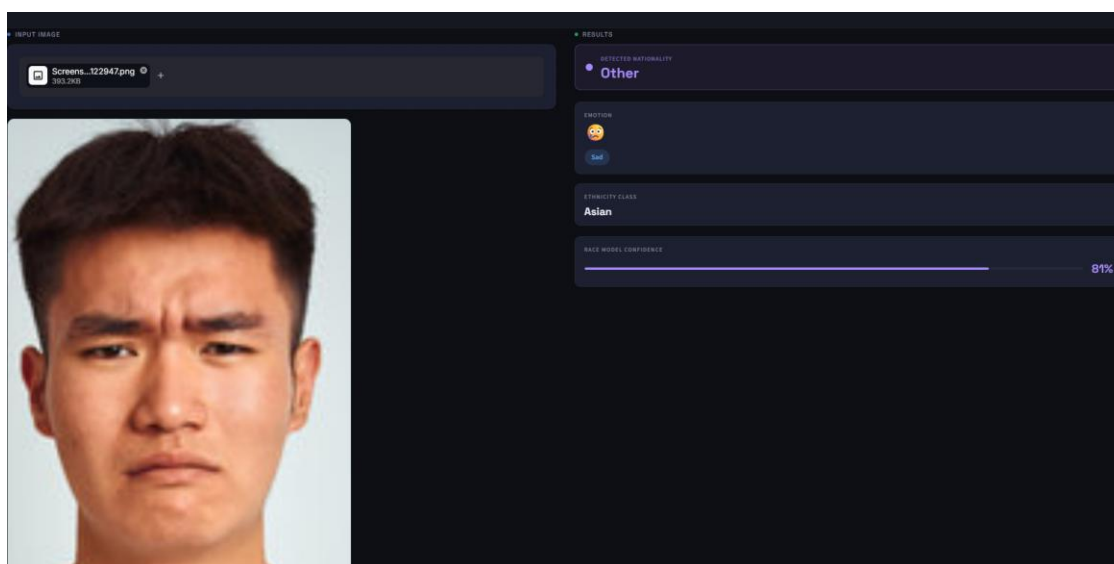


Figure 6 — another branch, showing how the displayed attributes change with the rule that applies.

11. Challenges Faced

Nationality versus ethnicity

The task asks for nationality, but nationality has no visual signal — it's a legal status. The honest move was to classify apparent ethnicity instead and map it to the branches, and to state that limitation plainly rather than pretend the model reads citizenship.

Indian vs African confusion

These two classes were the model's most common mix-up. Rather than retrain, I added a skin-tone brightness tie-breaker that only activates when the model is genuinely unsure between those two specific classes. It's a targeted patch on a known weakness, not a replacement for the model.

Isolating clothing from skin and background

Naively clustering the region below the face picked up skin and background instead of clothing. Adding a YCrCb skin-pixel filter and dropping near-white background pixels, then scoring clusters by saturation as well as size, made the dress-colour result far more reliable.

Mixing a trained model with a pretrained library

The app runs my own Keras models alongside the DeepFace library in one process. Caching every model at start-up with `@st.cache_resource` and keeping each component's preprocessing separate kept the app responsive and stopped the different input formats getting crossed.

12. Results Summary

Component	Role	Result
Ethnicity (mine, Keras)	Routes to the correct branch	~76% test accuracy (5 classes); strong on White/Black/Asian, weak on "Others"
Emotion (DeepFace)	Emotion in every branch	Integrated pretrained model (7 emotions)
Age (mine, Task 1)	Age for Indian/US branches	Reused Task 1 CNN, ~6 yr error
Dress colour (classical CV)	Colour for Indian/African	KMeans + LAB naming, 17 colours
Whole system	Branch → conditional attributes	Correct routing, shown in the GUI

The ethnicity model routes images to the correct branch in the large majority of cases for clear, front-facing photos, and the skin-tone refinement noticeably reduces the Indian/African confusion. The conditional logic behaves exactly as the brief specifies, which is demonstrated directly in the running interface.

13. Conclusion and Future Work

Task 3 shows that a multi-branch prediction system can be built cleanly by decomposing the requirement into independent components and combining them with a small, readable decision function. The most important design lesson is that the branching logic itself is trivial once each component is in place — the real work is choosing the right tool for each sub-problem and being honest about which parts are trained and which are integrated.

Concretely: the ethnicity classifier and the age model are my own Keras networks (the age model reused from Task 1), the dress-colour detector is my own classical-CV routine, and the emotion component is DeepFace's pretrained model integrated through a safe wrapper. Stating that split clearly is deliberate — it's both accurate and the kind of distinction a reviewer will check against the code.

Future improvements would include training a dedicated emotion model rather than relying on DeepFace (so the whole pipeline is self-built), strengthening the weak "Others" ethnicity class with more data, replacing the Haar cascade with a deep face detector for harder angles, and making the dress-colour region adapt to where the body actually is rather than assuming a fixed band below the face.